

JUPITER · SECURITY REVIEW

Jupiter Yield Store Review.

A security review of Yield Store Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Jupiter	Yield Store Review	Guardian	July 21 – August 3, 2026	August 12, 2026

§ 00

Contents.

94 pages — overview, confidence ranking, scope, 47 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 08
	Main Review · 47 findings	P. 12
§ 05	Disclaimer	P. 93
§ 06	About Guardian	P. 94

§ 01

Overview.

Guardian reviewed Yield Store Review across a single review round. The final report includes 1 Critical, 6 High, 16 Medium, 12 Low, 12 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	jup-ag/yield-store
CLIENT	METHODOLOGY
Jupiter	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
Yield Store Review	efcd0c5aa5c7
PREPARED BY	
Guardian	
AUDIT WINDOW	
July 21 – August 3, 2026	
FINAL REPORT	
August 12, 2026	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	1	0	1	0	0
● High	6	0	6	0	0
● Medium	16	0	16	0	0
● Low	12	0	12	0	0
● Informational	12	0	12	0	0
All severities	47	0	47	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Yield Store Review in `jup-ag/yield-store`. The scope includes 5,694 source lines across 72 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope 1/2.

FILE	NSLOC	TOTAL
interface/src/instructions.rs	360	373
program/src/instructions/delegate/protocol_cpi.rs	324	371
interface/src/nav.rs	320	463
interface/src/state/vault.rs	308	426
program/src/utls/oracle.rs	284	398
program/src/instructions/curator/initialize_vault.rs	224	309
program/src/instructions/curator/fulfill_redeem.rs	206	254
program/src/instructions/curator/fulfill.rs	200	244
program/src/instructions/user/redeem_direct.rs	195	237
program/src/instructions/curator/crystallize_fees.rs	156	209
program/src/utls/pda.rs	154	232
program/src/instructions/user/subscribe_direct.rs	151	183
interface/src/state/delegate/state.rs	134	191
program/src/instructions/user/subscribe.rs	130	156
interface/src/state/escrow.rs	111	137
interface/src/state/delegate/protocols/jupiter_lend.rs	107	167
program/src/helpers/vault.rs	105	135
program/src/instructions/user/redeem.rs	105	130
program/src/utls/token.rs	104	170
program/src/instructions/permissionless/validate_aum.rs	102	122
program/src/instructions/user/claim_redeem.rs	94	115
program/src/instructions/curator/grant_revoke_delegate.rs	88	102
program/src/instructions/user/cancel_redeem.rs	88	110
interface/src/state/delegate/protocols/offerbook.rs	87	113
program/src/instructions/user/claim.rs	85	106
program/src/instructions/user/cancel.rs	84	101
interface/src/state/mod.rs	81	116
program/src/lib.rs	72	83
program/src/instructions/curator/price_opaque.rs	70	84
program/src/utls/mod.rs	68	107
interface/src/errors.rs	67	72
interface/src/events/redemption.rs	57	67
program/src/instructions/curator/update_vault.rs	54	143
interface/src/state/delegate/protocols/jupiter_swap.rs	47	82
program/src/utls/event.rs	47	66
interface/src/events/deposit.rs	46	54

Files in Scope 2/2.

FILE	NSLOC	TOTAL
interface/src/events/admin.rs	45	61
program/src/instructions/curator/policy/initialize.rs	44	53
program/src/instructions/curator/policy/set_paused.rs	43	52
interface/build.rs	42	54
program/src/helpers/escrow.rs	41	46
program/src/instructions/curator/set_paused.rs	41	51
interface/src/state/policy.rs	40	54
interface/src/events/mod.rs	39	55
program/src/instructions/curator/accept_curator.rs	38	48
interface/src/arguments/initialize_vault.rs	35	44
interface/src/events/aum.rs	35	41
program/src/instructions/cpi/transfer_hook_execute.rs	32	49
interface/src/constants.rs	30	111
interface/src/state/delegate/protocols/mod.rs	30	36
interface/src/events/initialize_vault.rs	25	40
program/src/instructions/curator/mod.rs	24	26
interface/Cargo.toml	23	28
program/Cargo.toml	21	24
interface/src/state/delegate/permissions.rs	20	27
program/src/instructions/cpi/event.rs	18	23
interface/src/arguments/update_vault.rs	16	24
interface/src/utils.rs	14	15
interface/src/arguments/price_opaque.rs	13	14
interface/src/lib.rs	12	17
program/src/instructions/user/mod.rs	12	13
program/src/instructions/curator/close_vault.rs	11	15
program/src/instructions/curator/policy/update.rs	7	11
program/src/instructions/mod.rs	6	7
interface/src/arguments/mod.rs	4	5
interface/src/state/delegate/mod.rs	4	6
program/src/instructions/curator/policy/mod.rs	4	19
program/src/helpers/mod.rs	3	4
program/src/instructions/cpi/mod.rs	3	4
program/src/instructions/delegate/mod.rs	2	3
program/src/instructions/permissionless/mod.rs	2	3
program/src/utils/metadata.rs	0	171

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Queued claims trigger indirect reentrancy	Logical Error	CRITICAL	ACKNOWLEDGED	p. 12
H-01	Offerbook Position Skipped From AUM	Logical Error	HIGH	ACKNOWLEDGED	p. 14
H-02	Keepers Are Slowly Drained Through Rent	Logical Error	HIGH	ACKNOWLEDGED	p. 16
H-03	Delegates Can Route Funds To Themselves	Validation	HIGH	ACKNOWLEDGED	p. 18
H-04	End-to-End LIQUIDATE Flow Does Not Work	DoS	HIGH	ACKNOWLEDGED	p. 20
H-05	Partial cancel mints unbacked recovery shares	Validation	HIGH	ACKNOWLEDGED	p. 21
H-06	Price delegate amplifies gross exposure changes	Logical Error	HIGH	ACKNOWLEDGED	p. 23
M-01	Watermark NAV ignores accrued liability	Logical Error	MEDIUM	ACKNOWLEDGED	p. 26
M-02	Share freeze misses authority rotation path	Validation	MEDIUM	ACKNOWLEDGED	p. 28
M-03	Final Queue Redemption Strands Fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 29
M-04	Fee Updates Affect Retrospectively	Logical Error	MEDIUM	ACKNOWLEDGED	p. 30
M-05	Redeem cancellation bypasses paused transfers	Validation	MEDIUM	ACKNOWLEDGED	p. 31

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-06	Fresh oracle snapshots can misprice redemptions	Logical Error	MEDIUM	ACKNOWLEDGED	p. 33
M-07	Auto folding misprices debt haircut positions	Validation	MEDIUM	ACKNOWLEDGED	p. 35
M-08	Direct burns poison vault performance fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 37
M-09	Pyth confidence ignored during user settlement	Logical Error	MEDIUM	ACKNOWLEDGED	p. 39
M-10	External burns reset live vault NAV	Logical Error	MEDIUM	ACKNOWLEDGED	p. 41
M-11	Fresh timestamp allows stale vault pricing	Logical Error	MEDIUM	ACKNOWLEDGED	p. 43
M-12	Zero AUM bypasses liquidation loss budget	Logical Error	MEDIUM	ACKNOWLEDGED	p. 45
M-13	Unpaid fees can permanently freeze vault	Logical Error	MEDIUM	ACKNOWLEDGED	p. 47
M-14	Opaque debt bootstrap collapses AUM pricing	Logical Error	MEDIUM	ACKNOWLEDGED	p. 49
M-15	Cross mint defaults corrupt asset ledgers	Logical Error	MEDIUM	ACKNOWLEDGED	p. 51
M-16	Offerbook recalls create phantom vault value	Logical Error	MEDIUM	ACKNOWLEDGED	p. 53
L-01	Missing Remaining Accounts Validation	Validation	LOW	ACKNOWLEDGED	p. 55
L-02	Same-Protocol Positions Break Accounting	Validation	LOW	ACKNOWLEDGED	p. 56
L-03	Incoherent Position Flags Allowed	Configuration	LOW	ACKNOWLEDGED	p. 57
L-04	Fluid Positions Have No Upper Limit	Validation	LOW	ACKNOWLEDGED	p. 58
L-05	Lender Only Offerbook Mode Is Not Enforced	Validation	LOW	ACKNOWLEDGED	p. 59

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-06	Misleading Event In Redeem Due To Stale Remainder	Events	LOW	ACKNOWLEDGED	p. 60
L-07	Jupiter Lend borrow fees inflate AUM	Logical Error	LOW	ACKNOWLEDGED	p. 61
L-08	Stale watermark recharges later depositor	Logical Error	LOW	ACKNOWLEDGED	p. 63
L-09	Liquidate can sell Fluid ownership token	Validation	LOW	ACKNOWLEDGED	p. 65
L-10	Slippage budget resets allow adjacent losses	Logical Error	LOW	ACKNOWLEDGED	p. 67
L-11	Retained exit fees misprice curator shares	Logical Error	LOW	ACKNOWLEDGED	p. 68
L-12	Tiny payment fixes entire redemption NAV	Logical Error	LOW	ACKNOWLEDGED	p. 70
I-01	Metadata Growth Needs Direct Rent Top-Up	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 72
I-02	Informational Note For Users Regarding Oracle	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 73
I-03	Fluid Positions Cannot Be Closed	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 74
I-04	Mint And Oracle Pair Can Mismatch	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 75
I-05	Uninitialized token mint enables share inflation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 76
I-06	Rounded remint clears unpaid redeem value	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 78
I-07	Pending redeems hidden by supply saturation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 80
I-08	Queued deposits settle against stale AUM	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 82
I-09	Borrowed args create unchecked Rust strings	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 84

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Duplicate account roles violate borrow safety	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 87
I-11	Bad feed exponent can stall redemptions	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 89
I-12	Share reentry wrongly charges performance fees	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 91

CRITICAL **C-01**

ACKNOWLEDGED

Queued claims trigger indirect reentrancy

CATEGORY	Logical Error
LOCATION	claim()
REVIEW	Main Review

DESCRIPTION

The vault share token is configured so every share transfer must call back into the Yield Store program first

The queued deposit path is supposed to work like this

1. User deposits base tokens into escrow using Subscribe
2. A delegate fulfills it using Fulfill
3. Fulfill moves the base tokens into vault custody, mints share tokens into an escrow share account, then marks the escrow as fulfilled
4. User calls Claim
5. Claim transfers the escrowed shares from escrow to the user

The problem is in step 5, Claim is already running inside Yield Store, and then it calls Token-2022 TransferChecked to move the shares

But because the share mint has a transfer hook, Token-2022 then tries to call Yield Store again. So the call stack becomes

```
Yield Store::Claim
└─ Token-2022::TransferChecked
└─ Yield Store::transfer_hook_execute
```

Solana does not allow this kind of indirect re-entry, direct program calls itself can be allowed, but "program A calls program B, which calls program A again" is rejected by the runtime with ReentrancyNotAllowed.

So a normal queued subscription can get stuck after fulfillment, claim will fail every time because of the indirect reentrancy,

Cancel cannot help, because cancel only accepts FULFILLED_PENDING, while Fulfill changed the escrow to FULFILLED_DONE

The vault holds the deposited assets and the escrow account holds the shares, but the user cannot access either without a program upgrade

C-01 Queued claims trigger indirect reentrancy

CONTINUED

— RECOMMENDATION

we need to not perform a hooked share transfer from inside the hook program, we could burn the escrow share balance, signed by the escrow PDA, mint the same number of shares directly to the user, signed by the vault-authority PDA, then close the escrow share account and escrow state

That preserves total supply and avoids invoking the transfer hook. alternatively, mint directly to the user during Fulfill

PROOF OF CONCEPT https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/f16a2938475c76cb78288af6eefd2e070afcac82/integration-tests/tests/guardian/pocs/claim_reentrancy.rs

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/29>

HIGH **H-01**

ACKNOWLEDGED

Offerbook Position Skipped From AUM

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/protocol_cpi.rs#L73 , https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/protocol_cpi.rs#L320-L325 , https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/permissionless/validate_aum.rs#L87-L90
REVIEW	Main Review

— DESCRIPTION

An Offerbook delegate can deploy vault-owned depositor funds by calling `ProtocolCPI -> escrow_token_deposit`, which transfers assets from the vault custody token account into the vault's Offerbook escrow. The expected accounting model is AUM-continuous: deployed funds should stop appearing as idle custody, but should be represented by the Offerbook position mark.

Normally, this fund deployment is followed by `PriceOpaque`, which marks the deployed Offerbook leg before `ValidateAum` refreshes NAV. However, this sequence is not enforced atomically and there can be a gap between `PriceOpaque` and Offerbook deployment. On a first deploy from a zero-valued position, the protocol clears `PRICED` and `INITIALIZED`, then folds the deployed value into `last_value` without restoring either flag, creating an inconsistent state where the position has nonzero value but is still treated as uninitialized.

Because `ValidateAum` is permissionless, anyone can call it while the vault is in this inconsistent state. The required Offerbook position is skipped, so the vault records a fresh AUM that excludes real deployed depositor funds. With partial deployment, this lets a regular user `SubscribeDirect` at an artificially low NAV and receive excess shares; with full deployment, it can force a false zero-AUM state and block the honest first `PriceOpaque`.

— RECOMMENDATION

After any settlement fold that makes `last_value > 0`, set `INITIALIZED` while leaving `PRICED` unset, so early `ValidateAum` fails closed with `PricedSlotMissing` until a valid `PriceOpaque` completes. Alternatively, enforce deploy and first pricing atomically

PROOF OF CONCEPT <https://github.com/GuardianOrg/yield-store-team1-1784049631897/pull/1>

— RESOLUTION

Status: Acknowledged.

H-01 Offerbook Position Skipped From AUM

CONTINUED

<https://github.com/jup-ag/yield-store/pull/30>

HIGH **H-02**

ACKNOWLEDGED

Keepers Are Slowly Drained Through Rent

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/fulfill_redeem.rs#L158-L164 , https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/fulfill.rs#L218-L225
REVIEW	Main Review

DESCRIPTION

When fulfilling either a queued subscription or queued redemption, the **FULLFILL** authority, normally an automated keeper, pays the rent for a temporary token account. A subscription creates a temporary share account costing approximately 0.00208104 SOL, while a redemption creates a proceeds account costing approximately 0.00203928 SOL. When the user claims, the temporary account is closed and its rent is transferred to the user rather than returned back to the keeper.

This happens during regular protocol activity. With hundreds of users it slowly drains the keeper without requiring an attack. But it can also be amplified by splitting deposits or shares into many dust or small queued requests. The keeper's rent cost is fixed and unrelated to the request's value, an automated keeper that fulfills every valid request can be deliberately drained until it can no longer fund fulfillment transactions, potentially disrupting queued protocol operations.

At approximately \$74.33 per SOL with the current price, each fulfillment costs the keeper about \$0.15; 10,000 fulfillments would cost approximately 20.4–20.8 SOL, or \$1,516–\$1,547, excluding transaction fees. If SOL returns to its previous all-time-high price in the next market cycle, keeper will lose approximately \$0.60 in rent just for a single queued fulfillment and it will cause continuous value leak.

RECOMMENDATION

Either collect the temporary-account rent from the user when the request is queued and use the escrow PDA to fund fulfillment, or record the keeper that paid the rent and return it to that keeper when the account closes.

PROOF OF CONCEPT <https://github.com/GuardianOrg/yield-store-team1-1784049631897/pull/3>

RESOLUTION

Status: Acknowledged.

H-02 Keepers Are Slowly Drained Through Rent

CONTINUED

<https://github.com/jup-ag/yield-store/pull/43>

HIGH **H-03**

ACKNOWLEDGED

Delegates Can Route Funds To Themselves

CATEGORY Validation

LOCATION https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/interface/src/state/delegate/protocols/jupiter_lend.rs#L58-L86

REVIEW Main Review

— DESCRIPTION

Fluid checks position ownership when withdrawing collateral or borrowing assets. However, it doesn't check when adding collateral or repaying debt and intentionally allows anyone to repay another position's debt.

The `jupiter_lend.validate` pins the recipient and validates accounts such as `OPERATE_RECIPIENT_INDEX`, `OPERATE_RECIPIENT_BORROW_TA`, and `OPERATE_RECIPIENT_SUPPLY_TA`. These validations correctly restrict which account gets the borrowed or withdrawn funds. However, it doesn't validate which positions/accounts are credited during collateral additions or repayments.

Collateral deposits are funded from `OPERATE_SIGNER_SUPPLY_TA`, which is `cpi[1]`, but they are credited to delegate-provided position at `cpi[11]/cpi[12]`. The credited fluid position at `cpi[11]` or its ownership token account at `cpi[12]` are never validated.

Noting that the expected flow is initializing a position for the Yield vault, and using that position during operate calls. However, that position identity is neither recorded during initialization nor enforced during subsequent operate calls.

As a result, a delegate can provide its own Fluid position in `cpi[11]/cpi[12]` while supplying real vault assets from Yield's supply token account. Fluid accepts the collateral addition or debt repayment, and Yield records the outflow as deployed value belonging to its position. Yield cannot later withdraw those assets because its vault authority does not own the Fluid position, while the delegate can call Fluid directly to withdraw the collateral, leaving Yield with lost custody and stale deployed and AUM accounting.

— RECOMMENDATION

Record the vault-controlled Fluid position and position token account during initialization and require every subsequent operate call's `cpi[11]` and `cpi[12]` to match those stored accounts and remain controlled by the vault authority.

H-03 Delegates Can Route Funds To Themselves

CONTINUED

If multiple Fluid positions are expected, maintain a per-vault registry of curator-approved position pairs and require each supplied `cpi[11]/cpi[12]` pair to match an active registry entry.

PROOF OF CONCEPT <https://github.com/GuardianOrg/yield-store-team1-1784049631897/pull/4>

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/47>

HIGH **H-04**

ACKNOWLEDGED

End-to-End LIQUIDATE Flow Does Not Work

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/protocol_cpi.rs#L254-L256
REVIEW	Main Review

— DESCRIPTION

LIQUIDATE is separate from the normal **SWAP** path: SWAP exchanges listed vault assets, while **LIQUIDATE** is specifically designed to sell unlisted collateral seized from defaulted Offerbook loans. For this to work, the unlisted collateral must already be held in the vault authority's canonical ATA.

In the intended Offerbook recovery flow, defaulted collateral is first seized via **claim_token_loan** into the Yield-controlled Offerbook escrow. It must then be withdrawn from Offerbook escrow into the vault authority ATA before **LIQUIDATE** can swap it.

However, the **escrow_token_withdraw** settlement path requires the withdrawn mint to exist in the vault asset list. An unlisted asset cannot be withdrawn. So while the intention is to be able to seize and withdraw unlisted collateral, it can never be withdrawn from escrow to ATA, and is stuck in escrow.

Noting that the existing tests pass because they directly mint/fund the unlisted token into the vault authority ATA, bypassing the real **claim_token_loan** -> **escrow_token_withdraw** -> **LIQUIDATE** flow.

— RECOMMENDATION

Allow **escrow_token_withdraw** to withdraw unlisted seized collateral into the vault authority ATA when the withdrawal is tied to the liquidation-sink/claim recovery path, without folding it as a normal whitelisted deployed asset

PROOF OF CONCEPT <https://github.com/GuardianOrg/yield-store-team1-1784049631897/pull/5>

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/40>

HIGH **H-05**

ACKNOWLEDGED

Partial cancel mints unbacked recovery shares

CATEGORY Validation
LOCATION `cancel_redeem()`
REVIEW Main Review

— DESCRIPTION

The vault has three things, AUM: assets the vault has, `pending_redeem_value`: money already promised to redeemers, `shares`: ownership of whatever is left after those promises

So the live shareholders are only backed by

```
AUM - pending_redeem_value - fees
```

The code calls this `eff_aum`, But `Vault::eff_aum` uses saturating subtraction, so if the vault owes more than it has, it returns 0, that zero is dangerous

Normal deposits are protected: `deposit_basis` explicitly rejects `eff_aum == 0`, because `shares_for_deposit` would mint far too many shares

The problem is partial redeem cancellation skips that protection

In `cancel_redeem`, the code directly calls

```
shares_for_deposit(amount, eff_supply, eff_aum)
```

Then it subtracts the cancelled redeem remainder from `pending_redeem_value`, and mints those shares

cancelling a redeem remainder is only equivalent to a deposit if the vault is solvent

If the vault is underwater, removing a \$1,000 promise may not increase real free backing at all,

Example: vault has \$100k, owes \$110k, user cancels a \$1k remainder. The vault still owes \$109k, so live backing is still zero, but the user can receive a massive number of shares priced against zero backing

The attacker can turn a small cancelled redeem claim into a huge share position, then capture almost all future recovery if the vault later becomes solvent again

H-05 Partial cancel mints unbacked recovery shares

CONTINUED

Existing shareholders get heavily diluted, and the share supply can also be pushed near u64 limits, breaking later mints / deposits / cancellations

— RECOMMENDATION

Route partial cancellation through the same guarded basis as deposits

```
let (eff_supply, eff_aum) = vault.deposit_basis(share_supply, now)?;  
let shares = shares_for_deposit(amount, eff_supply, eff_aum)?;
```

Because a partial redeem necessarily has `pending_redeem_value > 0`, it cannot legitimately take the bootstrap branch

`deposit_basis` will therefore reject the dangerous zero-effective-AUM state.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/70>

HIGH **H-06**

ACKNOWLEDGED

Price delegate amplifies gross exposure changes

CATEGORY Logical Error
LOCATION price_opaque()
REVIEW Main Review

— DESCRIPTION

The drift guard is enforced independently against each `Vault.positions[i].last_value`

`credit_i change <= drift_cap * credit_i`

`debt_i change <= drift_cap * debt_i`

The implementation permits the same percentage movement both upward and downward

But `ValidateAum` does not settle each row independently, It computes

```
AUM = spot + sum(haircut-adjusted credits) - sum(debts)
```

A malicious PRICE delegate can move every credit row upward and every debt row downward within its individually valid drift budget

Every `PriceOpaque` call passes, but the resulting AUM increase is based on gross exposure

```
delta AUM = drift_cap * (gross credits + gross debts)
```

The intended security boundary should instead be related to net AUM

When credits and debts nearly offset, gross exposure can be many times larger than shareholder equity

So a small allowed movement in each row becomes an arbitrarily large percentage movement in NAV

Example, the vault has \$10.00 million cash, a \$100 million credit position, and a \$99 million debt position

The correct initial AUM is \$11.00 million

After one full default drift window, the PRICE delegate changes the marks to

```
Credit: $100.00m -> $102.00m
```

```
Debt: $99.00m -> $97.02m
```

H-06 Price delegate amplifies gross exposure changes

CONTINUED

Both rows move exactly 2%, so both pass the default per slot guard

ValidateAum then records

```
$10.00m + $102.00m - $97.02m = $14.98 million
```

A 2% movement per row has created a \$3.98 million, or 36.18%, increase in net AUM

With higher leverage, the percentage increase in NAV approaches unbounded values even though every individual mark remains inside its configured band

The vault has initialized required credit and debt rows with large offsetting gross values

The attacker controls or colludes with the vault PRICE delegate and owns shares

After the rows accumulate their drift budget, the delegate raises the credit marks and lowers the debt marks by the maximum permitted amounts

The attacker then calls permissionless ValidateAum, so the inflated AUM is fully fresh and no stale cache condition is needed

The attacker redeems shares against the inflated AUM through RedeemDirect or the queued redemption flow

The excess claim is paid from real idle custody, for a shareholder owning 50% of the example vault, the artificial component alone adds around \$1.99 million to the redemption claim before fees

The delegate can repeat the manipulation in later windows, they can also first create the opposite low NAV phase to acquire shares cheaply and then reverse the rows

Its actual vault level bound is proportional to gross credits plus gross debts, not shareholder equity, a colluding shareholder can extract real listed assets while leaving the remaining shareholders backed by overstated opaque credits and understated liabilities

— RECOMMENDATION

Make position pricing an atomic array level operation, apply the existing per-row checks, then compute the complete proposed net position value using the same debt and haircut semantics as ValidateAum

Reject the batch when the resulting vault level AUM or NAV change exceeds a configured aggregate limit

H-06 Price delegate amplifies gross exposure changes

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/71>

MEDIUM **M-01**

ACKNOWLEDGED

Watermark NAV ignores accrued liability

CATEGORY Logical Error
LOCATION nav_for_watermark()
REVIEW Main Review

DESCRIPTION

fee_liability correctly includes already folded performance fees

```
self.accrued_fee_assets(now)
.saturating_add(u64::from(self.perf_fee_accrued))
.saturating_add(self.perf_pending(eff_supply, now))
```

But nav_for_watermark, which decides whether another performance fee is due, does not subtract perf_fee_accrued

```
let aum_net = u64::from(self.aum)
.saturating_sub(u64::from(self.pending_redeem_value))
.saturating_sub(self.accrued_fee_assets(now));
```

It subtracts pending redemptions and streaming fees, but not already folded performance fees

Those fee assets are still physically inside the vault, but economically they no longer belong to normal shareholders. They are already owed to the fee recipient

When someone redeems, their shares are burned and they are paid using fee-net AUM. The fee assets stay behind. Now fewer shares remain, but the HWM math still counts those fee assets as if they belong to remaining shareholders

That makes NAV look higher even though the vault made no new profit, then the next accrue_fees() can say

NAV went up, charge performance fee again

But the NAV only went up because of accounting, not because of real investment gain

RECOMMENDATION

nav_for_watermark needs to account for already folded performance fee liabilities. Force that in the absence of investment P&L, a pure redemption must not increase perf_fee_accrued

M-01 Watermark NAV ignores accrued liability

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/34>

MEDIUM **M-02**

ACKNOWLEDGED

Share freeze misses authority rotation path

CATEGORY	Validation
LOCATION	transfer_hook_execute()
REVIEW	Main Review

— DESCRIPTION

The vault shares live inside Token-2022 token accounts. the pause is supposed to stop people from moving those shares to someone else

In the protection, the hook checks `Policy.paused_transfers`, and if it is set, it rejects the token transfer

The problem is that the hook only runs when there is a token transfer. It does not run when the owner of a token account is changed

A holder can do this

1. While transfers are allowed, put shares into a custom Token-2022 share account.
2. Make that account without the `ImmutableOwner` extension.
3. Later, after `paused_transfers` is enabled, do not transfer the shares.
4. Instead, call `Token-2022 SetAuthority(AccountOwner)` and change the token account owner to the buyer.

The shares never moved between token accounts, so the transfer hook is never called. But control of the account moved to the buyer, so in practice the buyer now controls the shares

So the pause does not fully freeze share ownership

— RECOMMENDATION

The hook need to reject transfers into share accounts that lack `ImmutableOwner`, alternatively, use a share account model in which every valid holding account is a canonical ATA and enforce that restriction in the transfer hook

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/33>

MEDIUM **M-03**

ACKNOWLEDGED

Final Queue Redemption Strands Fees

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/curator/crystallize_fees.rs#L75-L78
REVIEW	Main Review

— DESCRIPTION

Queued redemptions burn the user's shares immediately and add them to `pending_redeem_shares`, preserving the effective supply until settlement. When all outstanding shares are queued, the share mint's live supply becomes zero.

Although `crystallize_fees` calculates the effective supply including pending redemption shares, it returns early whenever the live mint supply is zero. Consequently, the fees cannot be minted to the curator and protocol while all shares are queued.

When all shares are queued and then fulfilled, payouts are correctly calculated based on net, but the fee assets stay in the vault and these remaining fees cannot be crystallized due to share supply being zero.

These assets then can be swept to any address without respecting the intended curator/protocol fee allocation because sweeping only checks live supply and pending redemptions but does not check fee liabilities. If the vault is reused instead of closed, old fee liabilities can affect the new generation of depositors.

— RECOMMENDATION

Allow fees to be crystallized using the effective supply when `pending_redeem_shares` is nonzero, even if the live mint supply is zero. Additionally, before a fulfillment reduces the effective supply to zero, require all outstanding fees to be crystallized or settle them atomically as part of the final fulfillment.

Also, consider requiring fee accumulators to be zero before vault sweeping and closure.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/44>

MEDIUM **M-04**

ACKNOWLEDGED

Fee Updates Affect Retrospectively

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/curator/update_vault.rs#L129-L136
REVIEW	Main Review

— DESCRIPTION

Management-fee updates are intended to apply only prospectively, and the update guard checks the stored `fee_capital_time` and `perf_fee_accrued` and require them to be zero, which are reset to zero when fees are crystallized.

However, `fee_capital_time` is only stored value and does not reflect the live interval after time passes. It still remains zero until another instruction folds the elapsed interval, even though the live capital-time returned by `fee_capital_time_at` is already increasing.

Because `update_vault` does not materialize this live interval before checking the stored field, the curator can change the fee while `fee_capital_time` still appears zero. The next fee calculation reconstructs the entire elapsed interval and multiplies it by the new current rate, causing the update to affect fees retrospectively.

Similarly, the `perf_fee_accrued` check can also be stale because it checks only performance fees already folded into storage and ignores a live `perf_pending` liability, which can exist after AUM rises above the high-water mark.

— RECOMMENDATION

Fold the live capital-time and pending performance fee before checking for unsettled liabilities, and require crystallization and the management-fee update to occur atomically so the new rate applies only from the update onward.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/45>

MEDIUM **M-05**

ACKNOWLEDGED

Redeem cancellation bypasses paused transfers

CATEGORY Validation
LOCATION `cancel_redeem()`
REVIEW Main Review

DESCRIPTION

The vault tries to stop share transfers with a transfer hook, but Redeem then CancelRedeem can move shares without performing a transfer

It burns shares from one account, then mints the same shares into another account chosen by the caller

The transfer hook never sees that the share mint is configured with this program as its Token-2022 transfer hook

The hook rejects transfers when `Policy.paused_transfers != 0`

The problem is that the redeem cancellation path does not use a transfer

Redeem burns shares from `user_share_account`, then records only owner: user

The Escrow state has no original share account field

Later, CancelRedeem validates only the escrow state, type, owner, and vault, It then mints to the supplied `user_share_account`

There is no check that this token account belongs to the escrow owner, and no check that it is the same account from which the shares were burned

Alice can do

transfers paused

Alice Redeem: burn Alice shares

Alice CancelRedeem: mint those shares into Bob's share account

Economically, Alice moved shares to Bob while transfers were paused

The code did not call `TransferChecked`, so the transfer hook pause did not run, the delegate version is also real

M-05 Redeem cancellation bypasses paused transfers

CONTINUED

Token-2022 burn authority can be the token-account owner or an approved delegate

If Victim previously approved Eve as a delegate, Eve can burn Victim's shares through Redeem

The escrow owner becomes Eve, and Eve can cancel and remint the shares to Eve's own account

This turns an old token approval into share theft during an emergency pause, this breaks paused_transfers

It lets users route shares to another holder while the curator believes transfers are frozen, and lets approved token delegates exfiltrate shares if they still have allowance

It does not directly inflate the supply or drain vault TVL by itself because shares are burned and then reminted

But it is still serious because emergency transfer pause and compliance controls become bypassable, the PDA checks authenticate the escrow, not the destination account

— RECOMMENDATION

Store the original source share account in Escrow and only remint back to that account

Alternatively, reject delegate-initiated redeem by requiring the source token account owner to equal user

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/46>

MEDIUM **M-06**

ACKNOWLEDGED

Fresh oracle snapshots can misprice redemptions

CATEGORY Logical Error
LOCATION `validate_aum()`
REVIEW Main Review

— DESCRIPTION

The vault says its total value was priced this second, but it does not remember which exact oracle prices were used

An attacker can use one valid Pyth price for the vault's AUM, then a different valid Pyth price for depositing, then another for redeeming, as long as all of them are fresh enough

`validate_aum` accepts caller-provided oracle accounts and calculates the total vault value, but stores only, `vault.aum`, `vault.last_priced_at`

`require_priced_now` checks only the same Unix second, not the same oracle price or update

Oracle reads accept any fresh update for the configured feed ID

`subscribe_direct` reads its own caller-supplied price again

`redeem_direct` also reads its own caller-supplied price again

If one fresh price says \$100 and another fresh price says \$101, the attacker can deposit using the higher price to mint more shares, then redeem using the lower price to receive more tokens back

If the price gap is bigger than the instant redeem fee, the round trip becomes profitable

This can cause direct loss of vault custody, with enough flash liquidity and enough idle tokens in the vault, the attacker can drain a large part, potentially all, of the idle balance

Existing protections reduce the window but do not fully stop it, there is a 10-second oracle freshness clamp, confidence checks, deposit caps, pause controls, idle-liquidity checks, and an instant redeem fee

The code itself says the 10-second clamp bounds, but does not eliminate, the sandwich

M-06 Fresh oracle snapshots can misprice redemptions

CONTINUED

The bug is valid when direct subscribe and redeem are enabled for an asset whose valid fresh oracle prices can move enough inside that freshness window

— RECOMMENDATION

Bind direct deposits and redemptions to the exact oracle observation used for AUM

Alternatively, recompute AUM and the user operation from one coherent price snapshot

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/48>

MEDIUM **M-07**

ACKNOWLEDGED

Auto folding misprices debt haircut positions

CATEGORY	Validation
LOCATION	initialize_vault()
REVIEW	Main Review

— DESCRIPTION

This happens when a folded protocol position is configured as debt or with a haircut

The vault has two buckets, real tokens sitting in vault custody, such as USDC in the vault wallet, and external position marks, meaning the vault believes its position in a protocol is worth a certain amount

when money moves between those two buckets, protocol_cpi uses a shortcut called folding

money leaves custody: increase position mark

money comes back to custody: decrease position mark

That shortcut is correct only if the position mark means a full-value positive asset

The problem is that vault initialization allows the same position row to mean something else

debt: validate_aum subtracts the mark

haircut: validate_aum counts only part of the mark

Those meanings are incompatible with the generic fold

If the vault borrows \$1M, custody receives \$1M, but the folded debt mark goes down

Then validate_aum sees both more cash and less debt, so reported AUM can rise by up to \$2M even though true equity did not increase

That inflated AUM feeds subscribe_direct, redeem_direct, queued fulfill paths, and fees

If \$1M is deployed into a position with a 1% haircut, reported AUM drops by \$10k even though true equity is unchanged

An attacker can subscribe during that artificial low NAV

M-07 Auto folding misprices debt haircut positions

CONTINUED

Then the delegate recalls the funds, AUM rises by the same deterministic amount, and the attacker redeems at a higher NAV

Existing holders absorb the loss

initialize_vault accepts debt flags and haircuts up to 10,000 bps, protocol_cpi folds without checking debt() or haircut_bps

Jupiter Lend Operate even re-stamps the position as priced immediately after folding

validate_aum then trusts that mark and applies debt or haircut semantics

Fresh oracle and ATA checks only prove the token movement amount, not that the accounting model is compatible

A configured or misconfigured vault can publish the wrong NAV

This can enable over minting shares, inflated redemptions, NAV timing arbitrage, and value transfer from honest holders to a colluding or compromised delegate and depositor

It is not fully permissionless, but it is triggerable under accepted configurations

— RECOMMENDATION

For every automatically folded protocol row, enforce at initialization

```
enabled = true
required_in_aum = true
debt = false
haircut_bps = 0
one unique row per protocol
```

If debt or haircut semantics are required, maintain separate gross collateral and debt marks with sign-aware updates, or require complete repricing instead of applying the generic fold

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/49>

MEDIUM **M-08**

ACKNOWLEDGED

Direct burns poison vault performance fees

CATEGORY Logical Error
LOCATION `accrue_fees()`
REVIEW Main Review

— DESCRIPTION

Those two must stay in sync in the vault

Vault.aum: how much money the vault has, share_mint.supply: how many shares exist

The vault assumes the share supply only changes through its own instructions

That is wrong because a user who owns shares can call Token-2022 BurnChecked directly and destroy their shares without going through Yield Store

Vault::nav_for_watermark calculates NAV as roughly vault money / share supply

Vault::accrue_fees updates hwm_nav when NAV goes above the old high-water mark

CrystallizeFees calls accrue_fees using the current Token-2022 mint supply

SubscribeDirect treats share_supply == 0 as bootstrap and ignores the existing AUM

An attacker can deposit into an empty vault and become the only shareholder

They can then burn almost all their shares directly through Token-2022

The vault still has the same AUM, but the supply is now tiny, the vault thinks NAV per share became huge, even though no real profit happened

CrystallizeFees records that huge fake NAV as the high-water mark

The fee shares round down to zero, but perf_fee_accrued is still cleared

The attacker burns the last share, because the supply is now zero, the next deposit bootstraps 1:1 even though old AUM is still there

The attacker can recover almost all their capital, but the fake HWM stays

The zero-supply guard only protects when supply is already 0, but the attack updates the HWM while supply is 1

pending_redeem_shares only tracks burns made through Yield Store Redeem, direct Token-2022 burns bypass it

M-08 Direct burns poison vault performance fees

CONTINUED

Transfer hooks pause transfers, not burns

sweep_idle and close_vault are curator teardown tools, they do not reset the poisoned HWM and cannot stop an atomic final burn plus re-bootstrap

Future depositors use a vault whose real NAV may be normal, but whose HWM is impossibly high

The protocol will not collect performance fees again until NAV rises above that fake level, which can be made arbitrarily large, this is an accounting bug

— RECOMMENDATION

we need to not clear an asset-denominated fee unless value was actually delivered, alternatively, accumulate fee dust until at least one raw fee share can be issued

and not advance hwm_nav merely because perf_pending > 0, advance it only after successful realization of the associated fee

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/50>

MEDIUM **M-09**

ACKNOWLEDGED

Pyth confidence ignored during user settlement

CATEGORY Logical Error
LOCATION `read_oracle_price()`
REVIEW Main Review

— DESCRIPTION

Pyth gives the program a price like token-a = $\$1.00 \pm \0.01

That does not mean the token is safely worth exactly \$1.00, It means the real market price may be somewhere around that range

But the vault checks that the uncertainty is not too large, then throws away the uncertainty and treats \$1.00 as an exact executable price

oracle.rs reads price and conf, uses conf only in gate_price, then returns only price and exponent

The confidence never reaches the deposit or redeem math, `subscribe_direct` values the user's deposited token at the midpoint and mints shares from that value

`redeem_direct` then lets the user burn those shares and withdraw another asset, again priced at the midpoint, minus only the instant redeem fee

The attacker buys token-a in the real market for \$0.99

Pyth says $\$1.00 \pm \0.01 , which passes the vault's 1% confidence cap

The vault credits the deposit as \$1.00, then lets the attacker instantly redeem into USDC with a 0.5% fee, receiving about \$0.995

The attacker paid \$0.99 and receives \$0.995, so the difference comes from vault liquidity

The existing checks reduce the blast radius but do not fix the bug

Same second AUM checks prevent stale or mixed-price attacks, but this attack can use one fresh coherent oracle update

`max_deposit`, idle liquidity, pause, disabling instant redeem, lowering confidence caps, or setting higher exit fees can limit or avoid it operationally

But the code does not enforce conservative pricing

M-09 Pyth confidence ignored during user settlement

CONTINUED

For an executable vault, deposits should normally use the lower bound, and redemptions or liabilities should use the upper bound

Pyth's best practices say protocols offering executable prices should account for confidence intervals and discount toward the adverse side

— RECOMMENDATION

Use conservative confidence adjusted pricing

Value deposits using the lower bound and redemptions or liabilities using the upper bound instead of treating the midpoint as an exact executable price

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/48>

MEDIUM **M-10**

ACKNOWLEDGED

External burns reset live vault NAV

CATEGORY Logical Error
LOCATION `is_bootstrap()`
REVIEW Main Review

— DESCRIPTION

The program treats share supply = 0 as this is the first deposit ever

But Token-2022 lets a share owner burn shares directly outside Yield Store, That can make the supply zero while the vault still holds assets

Think of vault shares like receipts

If the vault has \$2,000 and there are 1,000 shares, each share is worth about \$2

The attacker temporarily gets all 1,000 shares

They burn those shares directly with Token-2022, not through Yield Store

The vault still has \$2,000, but the mint supply is now 0, Yield Store sees 0 shares and treats the vault as brand new

The attacker deposits \$3,000 and receives 3,000 new shares at \$1 each

Now the vault has \$5,000 and 3,000 shares, so each share is worth about \$1.67

The attacker returns 1,000 shares to the lender and keeps 2,000 shares

The lender receives the same number of shares back, but those shares are now backed by less value

`is_bootstrap` only checks share supply is zero, no pending redeem shares, no pending redeem value

Then `deposit_basis` returns (0, 0)

So the deposit math ignores the real vault AUM

The share mint only has a transfer hook and no freeze authority

The transfer hook only controls transfers, not burns

Yield Store's normal redeem path burns shares and records the accounting

M-10 External burns reset live vault NAV

CONTINUED

But a direct Token-2022 burn reduces the mint supply without updating that accounting

This can steal the vault's accumulated yield or NAV premium from lenders, AMMs, or holders whose shares are temporarily borrowed

The attack needs control of the full live share supply, so it is not always available

But it is realistic for concentrated vaults or vaults whose shares sit in one lending market or pool

The core mistake is confusing zero shares exist right now with this vault has never had depositors, those are not the same

— RECOMMENDATION

A zero live mint supply must not, by itself, reactivate genesis pricing

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/52>

MEDIUM **M-11**

ACKNOWLEDGED

Fresh timestamp allows stale vault pricing

CATEGORY Logical Error
LOCATION `require_priced_now()`
REVIEW Main Review

— DESCRIPTION

The vault uses an old total-value number after something important has changed

The vault stores this in `Vault.aum`, deposits and redemptions use it to decide how many shares to mint, how much money to pay out

`validate_aum` reads custody token balances, position marks, debt flags, and haircuts, then writes

`vault.aum = ...`

`vault.last_priced_at = now`

The bug is that later code checks only the timestamp

`now == vault.last_priced_at`

That proves only that AUM was calculated sometime during this Unix second

It does not prove that nothing changed after AUM was calculated

`validate_aum` calculates AUM at time `T`

During the same second `T`, a delegate or curator changes something that affects AUM

`vault.last_priced_at` is still `T`

`subscribe_direct`, `redeem_direct`, `fulfill`, or `fulfill_redeem` accepts the cached AUM as fresh

Shares or payouts are calculated using the old value

The vault has 1,000 shares and `AUM = $1,000`

Each share is worth \$1, AUM is validated

Then, during the same second, a \$400 position is written down, haircutted, or invalidated

Real AUM should now be \$600

M-11 Fresh timestamp allows stale vault pricing

CONTINUED

But cached Vault.aum is still \$1,000 and last_priced_at still matches now

A user redeems 100 shares, they receive about \$100 instead of the correct \$60

The extra \$40 comes from the remaining vault users, the same works in the other direction

If real AUM goes up but cached AUM stays low, a depositor can mint too many shares cheaply and dilute existing users

price_opaque changes position.last_value but does not invalidate Vault.last_priced_at

protocol_cpi can clear a position's PRICED flag, meaning a new validate_aum would fail or require repricing, but same-second settlement can still use the old vault AUM

protocol_cpi can write down liquidation-sink marks without invalidating the aggregate vault AUM

update_vault can change asset oracles, can also change position haircuts

Oracle freshness checks only prove the oracle data is fresh, and drift guards limit only some PRICE delegate mark changes

PRICED flags help future validate_aum calls, but they do not stop same-second direct settlement because settlement checks only Vault.last_priced_at

— RECOMMENDATION

Invalidate or version the AUM cache whenever any AUM input changes, A timestamp alone is weak

Increment an aum_inputs_version on every mark, configuration, or strategy change, store that version during validate_aum

Require both timestamp freshness and a matching version before deposits or redemptions

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/54>

MEDIUM **M-12**

ACKNOWLEDGED

Zero AUM bypasses liquidation loss budget

CATEGORY	Logical Error
LOCATION	charge_liquidation_slippage()
REVIEW	Main Review

— DESCRIPTION

A vault can hold \$100M of collateral and owe \$100M of debt, Its AUM is correctly \$0, even though the collateral is still real and valuable

For unlisted seized collateral, the vault cannot use an oracle price. Instead it uses a liquidation sink mark, if the mark says the collateral is worth \$100M, a liquidation is allowed to return as little as \$95M (a permitted 5% discount)

This is intentional to permit real world liquidation slippage, the 5% rule

There is supposed to be a second safety rule, total liquidation loss is limited to 10% of AUM per hour

If AUM is \$0, that budget should be \$0, so a sale with any loss should fail

But after `validate_aum` correctly calculates and records a fresh zero AUM, it also records a fresh timestamp,

The budget function then only checks `aum = 0` and returns success immediately, It mistakes freshly checked and worth zero net for never checked

For example, a compromised LIQUIDATE delegate can sell collateral marked at \$100M to its own pool for \$95M

The vault receives \$95M, the attacker receives collateral worth about \$100M, and captures about \$5M

The code permits that 5% shortfall (floor calculation), then skips the cumulative loss budget

The attacker needs a privileged LIQUIDATE key, a fresh nonzero liquidation sink mark, and an actual zero / negative net AUM state

It also cannot normally be repeated immediately because each liquidation must sell the entire source balance and clears the sink mark,

M-12 Zero AUM bypasses liquidation loss budget

CONTINUED

remarking it at zero AUM is capped at zero (marking restriction), But one 5% loss can still be very large and worsens creditor recovery or any later equity recovery

— RECOMMENDATION

distinguish unvalidated state with a boolean or using `last_priced_at = 0` and enforce zero slippage for validated zero AUM vaults

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/56>

MEDIUM **M-13**

ACKNOWLEDGED

Unpaid fees can permanently freeze vault

CATEGORY Logical Error
LOCATION `accrue_fees()`
REVIEW Main Review

DESCRIPTION

In performance fees, the promised cut of profit for the protocol or manager

Crystallizing means paying that promised cut by minting fee shares

After a profit, `accrue_fees` can record the fee as a fixed numeraire amount without paying it immediately

For example, it can record that the vault owes \$100 in fees

But if the investments later crash, `validate_aum` lowers AUM without lowering that already-recorded fee liability

Example, after a large gain, the vault has \$1,100 of assets, a \$100 fee liability, and \$1,000 left for shareholders

After a crash, the vault has only \$50 of assets but still owes the same \$100 fee liability

The value left for shareholders becomes zero, not negative \$50

The program deliberately converts negative shareholder backing into zero

`eff_aum` subtracts redemption promises and fees from AUM using saturating subtraction

This blocks deposits because a deposit at zero backing could mint an unfair number of shares

`deposit_basis` therefore rejects when effective AUM is zero

The contradiction is that the intended recovery action, `CrystallizeFees`, also refuses to run when nothing remains after fees

`CrystallizeFees` returns `InsufficientBacking` before minting fee shares or clearing the stored fee liability

There is no other instruction that waives, reduces, writes down, or settles this fee debt

New normal deposits fail even when a depositor wants to rescue the vault

M-13 Unpaid fees can permanently freeze vault

CONTINUED

A rescue requires an out of band asset transfer or a market recovery before normal deposits can work again. Instant withdrawals revert because `redeem_direct` requires the post withdrawal backing to remain positive. Queued withdrawals are valued as though shareholder backing is zero, aside from negligible virtual offset dust in `fulfill_redeem`.

If the assets later recover only slightly above the fee liability, crystallization can transfer almost the entire recovery to the fee recipients.

This state can happen after a genuine market loss.

— RECOMMENDATION

Choose explicitly who bears losses on uncrystallized fees; the safer model is to mint or store fee shares when the fee is earned.

Those fee shares will then gain and lose value alongside normal shareholders; alternatively, if fees remain fixed numeraire liabilities, cap or write down the unpaid fees when the vault assets cannot cover them.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/55>

MEDIUM **M-14**

ACKNOWLEDGED

Opaque debt bootstrap collapses AUM pricing

CATEGORY Logical Error
LOCATION price_opaque()
REVIEW Main Review

DESCRIPTION

The vault uses AUM to decide how much ownership a deposit receives

The first manual price for a position is limited only by value $\leq$ current AUM

That rule appears safe for a credit position because credit increases vault value

But debt has the opposite effect, ValidateAum subtracts debt marks from AUM

So the code limits the raw number without limiting what that number does to the vault's value

For a debt position, a large but allowed first mark can make a healthy vault appear almost worthless in one step

price_opaque skips drift checks on the first mark, It also does not prevent the generic pricing path from being used for debt positions

validate_aum skips a required position while it is still uninitialized, the vault can therefore first record its full custody value

The PRICE delegate can then initialize the debt position with a mark almost equal to that full AUM

A second ValidateAum subtracts the debt and records near-zero AUM, deposits are rejected only when effective backing is exactly zero

They are still allowed when the backing is only one raw unit, subscribe_direct then mints shares using that crushed AUM

Example: the vault contains \$1,000,000 and has 1,000,000 existing shares

An enabled required debt position has never been priced

ValidateAum skips that position and records the full \$1,000,000 AUM

A malicious PRICE delegate then sets the first debt mark to almost \$1,000,000

M-14 Opaque debt bootstrap collapses AUM pricing

CONTINUED

The mark passes because it is not greater than the validated AUM

The bootstrap path skips the normal drift restriction, ValidateAum then records almost zero vault value

The attacker deposits a tiny amount at this artificial low valuation, Because shares are priced against almost zero backing, the attacker receives almost all newly represented ownership

The delegate later reduces the fake debt mark over the allowed drift windows, As the fake liability disappears, the real \$1,000,000 custody value returns to AUM

But the attacker now owns almost all the shares, they can then redeem most of the real assets, existing shareholders are massively diluted and can lose most of the vault

The attack requires an enabled required in AUM debt position, an uninitialized mark, and control or collusion with the PRICE delegate

— RECOMMENDATION

Block debt positions from using the generic PriceOpaque bootstrap path, require their first value to come from authenticated protocol state or a special initialization process

During debt initialization, pause deposits and redemptions until the first mark is safely established

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/57>

MEDIUM **M-15**

ACKNOWLEDGED

Cross mint defaults corrupt asset ledgers

CATEGORY	Logical Error
LOCATION	protocol_cpi()
REVIEW	Main Review

— DESCRIPTION

deployed is a per mint record of how many tokens the vault currently has outside its custody

For example, when the vault sends out 1,000 USDC, USDC.deployed records that 1,000 USDC is still external

When the same USDC returns, the code removes that amount from USDC.deployed

This works only when the same token mint returns

The vault stores this accounting separately for every mint inside AssetConfig.deployed

During a normal Offerbook default, the vault can send out one mint and receive a different mint as collateral

The vault sends USDC to Offerbook and the code increases USDC.deployed

The borrower defaults and Offerbook gives the vault SOL collateral instead of USDC

claim_token_loan only marks the general Offerbook position as requiring a new price

It does not move the deployed accounting from USDC to SOL

When SOL is later withdrawn, the generic protocol_cpi code only sees that SOL arrived

It subtracts the returned amount from SOL.deployed

But SOL.deployed is already zero, because the subtraction cannot go below zero, it remains zero

The original USDC.deployed entry remains stuck

The accounting therefore says USDC is still deployed even though it no longer exists inside Offerbook

It also fails to record that SOL was held externally after the default

M-15 Cross mint defaults corrupt asset ledgers

CONTINUED

The system has no loan-level record connecting the returned SOL collateral to the original USDC principal

The settlement format only knows the token account moving during the current CPI

The vault can permanently lose USDC capacity

Future USDC subscriptions or deployments can be rejected because the stale USDC.deployed amount remains included in the cap calculation

The SOL capacity can be bypassed while the seized SOL remains inside Offerbook, new SOL deposits can be accepted without counting the external SOL exposure

The recovered SOL can later be withdrawn into custody, causing total SOL exposure to exceed the intended limit

— RECOMMENDATION

Track deployments per Offerbook loan or introduce a conversion settlement

containing source principal mint and amount, returned collateral mint and amount, the authenticated Offerbook loan or claim identity

on default, atomically reduce mint A's principal deployment and record mint B's collateral exposure

on withdrawal, reduce the mint B exposure associated with that exact loan or collateral lot

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/58>

MEDIUM **M-16**

ACKNOWLEDGED

Offerbook recalls create phantom vault value

CATEGORY Logical Error
LOCATION `protocol_cpi()`
REVIEW Main Review

— DESCRIPTION

The Offerbook position mark represents the estimated value of tokens and lending positions held outside the vault

When tokens return from Offerbook, value should move from the Offerbook bucket into custody without changing the total

This only works when both buckets use the same token price, Here they use different price bases

The Offerbook mark may say that 100 tokens were worth \$100 at an earlier time

The token price later falls to \$0.50

The bot recalls all 100 tokens

The code adds their current \$50 value to custody

But it also subtracts only the current \$50 value from the old \$100 Offerbook mark

The vault then records

$\$50 \text{ custody} + \$50 \text{ remaining Offerbook mark} = \100

But it owns only 100 tokens currently worth \$50

The root cause is that Offerbook withdrawal is declared as `Flow::In` with `dirty_tracked: false` inside `offerbook.rs`

`protocol_cpi` subtracts the recalled tokens live Pyth value from the historical position mark

But it does not clear `PRICED` or invalidate the position

`ValidateAum` then prices custody using the current oracle price and adds the remaining historical position mark

A shareholder can redeem against this inflated AUM and receive too many real tokens

M-16 Offerbook recalls create phantom vault value

CONTINUED

The remaining shareholders are left backed by fake external value

When instant redemption is enabled and the recalled tokens are idle, the sequence can be atomic recall -> ValidateAum -> RedeemDirect

The shareholder does not necessarily need control of the ESCROW delegate

They can capitalize after an honest bot performs the recall because ValidateAum is permissionless and RedeemDirect only requires the shareholder's own signature

The issue matters most for volatile assets or during a stablecoin depeg

— RECOMMENDATION

Invalidate the tracked Offerbook position after every withdrawal, clear PRICED and block ValidateAum until the complete remaining Offerbook state is authoritatively revalued

Alternatively, setting the position mark to zero is safe only when the program can prove that the entire external position, including active loans and accrued interest, has been fully closed

Note, don't simply update the mark timestamp, that would make the incorrect historical basis residual appear freshly validated

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/59>

LOW **L-01**

ACKNOWLEDGED

Missing Remaining Accounts Validation

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/curator/initialize_vault.rs#L58-L62
REVIEW	Main Review

DESCRIPTION

The `initialize_vault` instruction derives the vault's asset list from the remaining accounts passed to the instruction. However, it does not validate that at least one asset account is provided.

As a result, the curator can initialize a vault with an empty asset list. Since vault assets are fixed at initialization, this leaves the vault unusable while still causing the curator to spend rent.

RECOMMENDATION

Require `initialize_vault` to receive at least one remaining asset account before creating the vault and share mint.

RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/36>

LOW **L-02**

ACKNOWLEDGED

Same-Protocol Positions Break Accounting

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/protocol_cpi.rs#L315
REVIEW	Main Review

— DESCRIPTION

`initialize_vault` accepts curator-provided position slots from the instruction arguments, but it does not reject duplicate active positions with the same protocol. As a result, a vault can be initialized with multiple enabled positions for the same protocol type, even though downstream accounting paths identify CPI-settled positions by protocol rather than by a unique slot.

If same-protocol duplicate positions are not intended, this is a low-severity configuration validation issue: an incorrect vault can be created and later require operational cleanup or redeployment. However, if the same-protocol positions are intended to be supported, later CPI accounting can dirty all matching positions while folding value into only the first match, causing broken or incorrect position accounting.

— RECOMMENDATION

Enforce uniqueness for active positions by protocol during `initialize_vault` and any position update path. Alternatively, if multiple same-protocol positions are a supported feature, add explicit slot targeting to downstream accounting logic instead of relying on protocol-only matching

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/32>

LOW **L-03**

ACKNOWLEDGED

Incoherent Position Flags Allowed

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/curator/initialize_vault.rs#L316
REVIEW	Main Review

DESCRIPTION

`initialize_vault` accepts curator-provided position flags but only masks runtime bits and validates basic numeric bounds. It does not reject semantically incoherent combinations such as `REQUIRED_IN_AUM` without `ENABLED` or `LIQUIDATION_SINK` without the required accounting flags.

This is primarily a curator configuration issue. A trusted curator can initialize a vault with unusable or inconsistent position slots, potentially causing later pricing, AUM validation, or liquidation flows to fail or behave unexpectedly. Because position flags are immutable after initialization, these mistakes may require operational recovery or vault redeployment.

RECOMMENDATION

Add explicit semantic validation for position flags during `initialize_vault`. Require valid role combinations and reject contradictory flags. Alternatively, clearly document the required flags and warn curators for misconfiguration risks.

RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/32>

LOW **L-04**

ACKNOWLEDGED

Fluid Positions Have No Upper Limit

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/interface/src/state/delegate/protocols/jupiter_lend.rs#L87-L92
REVIEW	Main Review

— DESCRIPTION

Yield allows a delegate with the broad LEND permission to call Fluid's `init_position`, and only verifies the vault-authority signer. It does not limit the number of Fluid positions or restrict their markets, while Fluid has no practical upper limit on the number of positions that can be created.

Each position requires the vault authority to pay rent for the position, mint, position ATA, and metadata accounts. A delegate can repetitively initialize positions and consume SOL.

— RECOMMENDATION

Consider adding a cap for the Fluid positions.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/60>

LOW **L-05**

ACKNOWLEDGED

Lender Only Offerbook Mode Is Not Enforced

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/interface/src/state/delegate/protocols/offerbook.rs#L100-L101
REVIEW	Main Review

— DESCRIPTION

The README and inline documentation describe Offerbook vaults as lender-only and state that borrower-side instructions are rejected. However, this restriction is not enforced on-chain: if the curator grants the undocumented Offerbook `BORROW` or `REPAY` permissions, `offerbook.rs` allows the delegate to call `fill_token_principal_offer` and `repay_token_loan` even though the comments state they are rejected by design.

These calls allow a lender-intended vault to borrow principal and repay debt. More importantly, debt repayment causes broken accounting in this case as the drift cap guard does not allow previous non-zero debt to be cleared after payment.

— RECOMMENDATION

Reject Offerbook `BORROW` and `REPAY` permissions to enforce the documented lender-only model. Also clearly document it for curators.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/62>

LOW **L-06**

ACKNOWLEDGED

Misleading Event In Redeem Due To Stale Remainder

CATEGORY Events

LOCATION https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/delegate/fulfill_redeem.rs#L218-L232

REVIEW Main Review

— DESCRIPTION

`FulfillRedeem` converts the owed numeraire amount into payout-token units, then converts the actual paid token amount back into numeraire. Because these conversions round down, `done` can be true while `remaining_numeraire` is still a small dust amount.

When this happens, the escrow is marked `FULFILLED_DONE`, but the emitted `RedeemFulfilled` event still reports `remaining_numeraire` without clearing. Also, the event does not include the fulfilled status. As a result, an entirely fulfilled redeem can be interpreted as a redeem with some remainder by offchain indexers/frontends.

— RECOMMENDATION

When `done == true`, set the remaining to 0. Additionally, consider including the fulfilled status in the event so consumers can distinguish completed redeems from partial fills.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/63>

LOW **L-07**

ACKNOWLEDGED

Jupiter Lend borrow fees inflate AUM

CATEGORY	Logical Error
LOCATION	protocol_cpi, Pre::Operate
REVIEW	Main Review

DESCRIPTION

The vault calculates AUM as tokens the vault holds directly

+ value of outside positions

• debts

For Jupiter Lend / Fluid, the code does not fully read the outside Fluid position after every borrow or repay

Instead, it uses a shortcut: if tokens moved into the vault, reduce the external position mark by that token value, and if tokens moved out, increase it by that value

protocol_cpi calculates token-account deltas, converts them with oracles, then does

position mark = old mark + outflows - inflows

It then marks the position as freshly priced

After that, validate_aum trusts the cached position mark and does not re-read Fluid collateral or debt

The problem is that Fluid borrowing is not equal to tokens received

Upstream Fluid code adds the borrow fee into the position debt, while only the principal is transferred out to the borrower

If the vault borrows \$100 with a \$1 fee

Vault receives: +\$100

Fluid debt increases: -\$101

Code records: only -\$100

Hidden loss: \$1

Then if the delegate repays about \$100

L-07 Jupiter Lend borrow fees inflate AUM

CONTINUED

`Vault pays out: -$100``Code increases mark: +$100``But the real fee debt still remains`

The code's mark returns near the old value, but the real Fluid position is worse

Repeating this creates phantom AUM, AUM that exists in Yield Store accounting but not in reality

`validate_aum` only checks that the cached mark is fresh, it does not verify Fluid's real debt

The oracle only prices the token amount that moved, not the borrow fee added inside Fluid

The Jupiter Lend adapter intentionally keeps AUM continuous by folding deltas immediately

BORROW permission limits who can trigger it, but the threat model says compromised hot delegate keys must not be able to extract value

Pause, timelock, and idle-liquidity limits can restrict the timing or amount, but they do not fix the wrong accounting

A malicious or compromised BORROW delegate, especially with a colluding shareholder, can inflate reported NAV relative to real assets

The shareholder can redeem at the overstated price, taking too much idle liquidity and leaving the remaining holders with the hidden Fluid fee debt, interest, or liquidation loss

— RECOMMENDATION

we need to stop treating custody deltas as the full position valuation

After Fluid operate, the vault must either read and value the complete Fluid position, including collateral, debt, borrow fees, and accrued interest, or mark the position dirty so `validate_aum` cannot include it until an independent full reprice happens

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/64>

LOW **L-08**

ACKNOWLEDGED

Stale watermark recharges later depositor

CATEGORY Logical Error
LOCATION `accrue_fees()`
REVIEW Main Review

DESCRIPTION

The vault forgets to close a tiny old profit window when the performance fee rounds to zero

Later, after a large depositor enters, the vault reuses that same old profit window but multiplies it by the much larger share supply

That wrongly treats part of the new depositor's principal as old profit

AUM means total vault value, shares are claims on the vault, NAV means value per share, and HWM is the last NAV level already settled for performance fees

Performance fees should only charge gains earned by the holders who were present during that gain
`perf_pending` computes

```
fee = (current NAV - HWM) * current supply * 10%
```

But it uses integer rounding, so a real positive gain can produce `fee = 0`

The core problem is that `hwm_nav` is updated only if `pending > 0`

If the fee rounds to zero, the HWM stays old

Using the pre-deposit supply is normally correct

`subscribe_direct` accrues fees using the old supply before adding the new deposit, and `fulfill` does the same for queued deposits

That is supposed to protect new depositors from paying for old gains

But if the old gain rounded to zero and the HWM was not advanced, that old gain remains open

Later, `crystallize_fees` calls `accrue_fees` with the now-large current supply, so the old NAV gap is charged against the new depositor too

L-08 Stale watermark recharges later depositor

CONTINUED

The old holder earned a tiny gain, the fee rounded to zero, the vault did not mark that gain as settled

A large depositor entered, the vault later charged the old tiny gain as if all the new shares had earned it

ValidateAum counts donated custody balances into AUM, but it does not settle the performance HWM, it only accrues capital time before writing AUM

Deposit caps can limit the size, but max_deposit = 0 means unlimited, Fresh oracle checks do not help because the attack can use genuine oracle prices

A victim can lose a large part of their deposit immediately through fake performance fees

With the numbers in the report, about \$90,908 can be minted as protocol fee shares from a \$1,000,000 deposit, around 9.09%

The attacker does not necessarily receive the funds directly, the shares go to the configured protocol treasury

— RECOMMENDATION

Advance hwm_nav whenever nav > hwm_nav, even if the rounded fee is zero

That forgives less than one raw unit of dust instead of allowing that dust to be multiplied by future deposits

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/65>

LOW **L-09**

ACKNOWLEDGED

Liquidate can sell Fluid ownership token

CATEGORY	Validation
LOCATION	protocol_cpi()
REVIEW	Main Review

— DESCRIPTION

The vault has one program wallet called the vault authority

The code uses that same authority for many different jobs: holding normal vault tokens, signing Jupiter swaps, and owning the Fluid / Jupiter Lend position token

That is the root problem, LIQUIDATE is supposed to sell seized collateral from a bad loan

But the code does not prove this token is the seized collateral, It only checks whether it is an unlisted token in the vault authority's canonical token account

That check is too broad, A Fluid position ownership token can also be an unlisted token owned by the same vault authority

So the liquidation path can accidentally treat the Fluid ownership token like seized junk collateral

The Fluid position token is like the title to the Fluid position, whoever owns that token can control the position in Fluid

If LIQUIDATE sells that token, the attacker does not just buy a random token, they buy control over the whole external Fluid position

The vault authority PDA is shared, ["authority", vault], It is not separated by role

ProtocolCPI marks that authority as signer for forwarded protocol calls

Jupiter Lend / Fluid also uses that same authority as signer for init_position and operate

The unlisted liquidation branch only checks the canonical ATA, full sale, destination asset, and liquidation-sink mark

It does not check the source mint / account against a recorded seized-collateral lot

The vault stores position marks, not Fluid position identity such as the position mint or position token account

L-09 Liquidate can sell Fluid ownership token

CONTINUED

A malicious or compromised LIQUIDATE delegate could sell the vault's Fluid ownership token for a small amount of USDC

The attacker could then use that token outside Yield Store to withdraw collateral or borrow against the Fluid position

The loss can be the whole Fluid position, not just the liquidation mark

It requires a live Fluid position, a valid liquidation-sink mark, an executable Jupiter route for the position token, and a malicious or compromised LIQUIDATE delegate

— RECOMMENDATION

Bind every liquidation to an authenticated collateral lot, explicitly protect authority bearing assets, and domain-separate authority PDAs

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/66>

LOW **L-10**

ACKNOWLEDGED

Slippage budget resets allow adjacent losses

CATEGORY	Logical Error
LOCATION	charge_swap_slippage()
REVIEW	Main Review

DESCRIPTION

The code describes a rolling one hour cap of 2% of AUM, but `charge_swap_slippage` implements a tumbling window,

once `now - swap_slippage_window_start >= 3600`, it discards the entire previous `swap_slippage_spent` value and starts again from zero

The constants explicitly claim that cumulative loss is limited to 2% of AUM per hour to contain a compromised SWAP delegate

A compromised delegate can exploit the boundary as follows

- 1) At t_0 , execute a harmless or zero-slippage swap, choosing the window anchor.
- 2) At $t_0 + 3599$, execute enough individually valid swaps to consume the full 2% budget.
- 3) At $t_0 + 3600$, the next swap resets `swap_slippage_spent` to zero
- 4) Immediately consume another 2% budget

For a vault with \$10 million AUM, the delegate can lose \$200,000 immediately before the boundary and another \$200,000 immediately after it, approximately 4% of AUM across adjacent transactions, while every per-hop and cumulative check succeeds

The listed source settlement path charges each oracle valued shortfall through exactly this function

RECOMMENDATION

Replace the single `window_start` spent tumbling counter with a genuine sliding window mechanism, such as timestamped sub buckets covering the preceding 3,600 seconds

RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/67>

LOW **L-11**

ACKNOWLEDGED

Retained exit fees misprice curator shares

CATEGORY	Logical Error
LOCATION	redeem_direct()
REVIEW	Main Review

— DESCRIPTION

An instant exit charges a fee, that fee is supposed to remain inside the vault and become the curator's ownership by minting curator shares

For a normal deposit, the code correctly calculates shares using the money that was inside the vault before the deposit

But during RedeemDirect, it first pays the user their withdrawal minus the fee, so the fee is already left inside the vault

It then calls the new-deposit share calculator and incorrectly treats that same fee as though it is being added again

Example: a \$1,000,000 vault, a 0.5% exit fee, and the exiting holder keeps 1 share

After the large exit, the vault contains the \$5,000 retained fee plus around \$1 backing the remaining share

The curator should receive around 5,000 shares, but the current code gives the curator around 1 share

The attacker's 1 retained share should remain worth around \$1, but instead becomes worth around \$2,500 before performance-fee accounting

So the curator should receive almost all of the \$5,000 fee, but the code gives the curator about one share instead

The attacker's retained share captures roughly half of the fee's value

The high-water mark makes the issue worse, it's supposed to identify real investment profit and charge a 10% protocol performance fee on it

Because the curator was underpaid in shares, the remaining share price jumps from around \$1 to around \$2,498 without any real investment profit

The code mistakes that accounting error for investment performance

L-11 Retained exit fees misprice curator shares

CONTINUED

It removes around \$500 as a performance fee liability, but the attacker can still later withdraw around \$2,249 from their originally \$1 retained share

A large holder can avoid around 45% of the configured instant-exit fee by keeping a small share balance and redeeming it later through the queued path

With a 10% instant fee on a \$1m exit, the attacker can recover around \$45,000

A small existing holder can also capture part of another user's large exit fee

The curator loses intended fee revenue, and the protocol charges a performance fee on fake profit

— RECOMMENDATION

The fee is already inside the post redemption AUM, so it must not also be treated as a new deposit when calculating the curator shares

Subtract fee_numeraire from the post-redemption AUM basis before calling shares_for_deposit, use checked subtraction, and preferably round the fee share result upward so the curator receives at least the retained fee value

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/68>

LOW **L-12**

ACKNOWLEDGED

Tiny payment fixes entire redemption NAV

CATEGORY Logical Error
LOCATION fulfill_redeem()
REVIEW Main Review

— DESCRIPTION

A redeemer has two economic states

Before the first fill, their burned shares remain in `pending_redeem_shares`, so the unpaid redemption continues sharing vault gains, losses, and fees

After a fill, the claim moves to `pending_redeem_value`, becoming a fixed-numeraire IOU

The vuln is that any nonzero first payment, even dust, converts the entire redemption from shares to a fixed claim

FulfillRedeem prices the full redemption but pays only $\min(\text{owed}, \text{available})$, so a tiny available balance triggers the conversion

All shares leave `pending_redeem_shares`, while the full priced value enters `pending_redeem_value`

Example: a \$100 vault where Alice owns 40%. Alice queues a redeem and receives a \$0.01 first fill. The system locks her entire claim at roughly \$40

If vault NAV then falls to \$50, Alice still holds a ~\$40 protected IOU, leaving remaining shareholders with only ~\$10 instead of their fair ~\$30 share. Future losses are shifted to them

The reverse is also possible, dust-fill a user at a temporarily low NAV, then let the vault recover. The user loses the recovery upside while remaining shareholders receive it

`pending_redeem_value` is a protected reserve, and cancellation does not reverse the original price lock, it converts the remaining fixed claim back into shares at the later NAV, potentially realizing the advantage or loss

This allows the delegate to decide who keeps future upside, who absorbs future downside, and who stops paying fees

That conflicts with the project stated trust model that a compromised hot key should not be able to change economic terms or extract value

L-12 Tiny payment fixes entire redemption NAV

CONTINUED

— RECOMMENDATION

we need to not convert the full share amount after an arbitrary partial first fill, either require the first fill to fully fund the redemption before fixing its value, or convert only the proportion of shares actually funded

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/69>

INFORMATIONAL I-01

ACKNOWLEDGED

Metadata Growth Needs Direct Rent Top-Up

CATEGORY Informational

LOCATION https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/curator/update_share_metadata.rs#L16

REVIEW Main Review

DESCRIPTION

`UpdateShareMeta` allows the curator to update the share mint's Token-2022 metadata fields, but if the new metadata value increases the mint account's required rent-exempt balance, the update can fail with an insufficient-rent error. The instruction does not include a payer account or perform a lamport top-up before invoking Token-2022 metadata updates.

A curator wants to update metadata with the larger value needs to send required lamports directly to the share mint before calling `UpdateShareMeta`.

RECOMMENDATION

Document this requirement clearly for curators. Optionally, add a payer/top-up path to `UpdateShareMeta` so metadata growth can be handled atomically.

RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/35>

INFORMATIONAL I-02

ACKNOWLEDGED

Informational Note For Users Regarding Oracle

CATEGORY	Trust Assumptions
LOCATION	https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/utls/oracle.rs#L41
REVIEW	Main Review

DESCRIPTION

Yield Store accepts Pyth updates with `Partial(5)` verification for all oracle reads. Full verification requires 2/3 of Wormhole guardians and Wormhole has 19 guardians. This reduces the required Wormhole guardian signatures from the normal 13-of-19 quorum to 5-of-19, enabling atomic updates but weakening the oracle's integrity threshold.

Noting that this is an intentional trust assumption rather than a protocol bug, but vault users should be aware of the reduced threshold.

RECOMMENDATION

Document the 5-of-19 oracle trust assumption for users.

RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/80>

INFORMATIONAL I-03

ACKNOWLEDGED

Fluid Positions Cannot Be Closed

CATEGORY Informational

LOCATION https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/interface/src/state/delegate/protocols/jupiter_lend.rs#L166

REVIEW Main Review

DESCRIPTION

The Jupiter Lend integration supports Fluid's `init_position` and `operate` instructions but not `close_position`. Positions initialized through Yield are owned by the vault-authority PDA, meaning they cannot be closed directly outside Yield, while the CPI dispatcher rejects the close instruction.

RECOMMENDATION

Document this behavior. If positions are not intended to be permanently open, add a controlled `close_position` path.

RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/82>

INFORMATIONAL **I-04**

ACKNOWLEDGED

Mint And Oracle Pair Can Mismatch

CATEGORY Trust Assumptions

LOCATION https://github.com/GuardianOrg/yield-store-team1-1784049631897/blob/efcd0c5aa5c72dc70cf4d3bb1cecd3ddd768d6e4/program/src/instructions/curator/initialize_vault.rs#L261-L268

REVIEW Main Review

— DESCRIPTION

The protocol verifies that each price update is genuine Pyth data and matches the feed ID configured by the curator. However, it does not verify that the feed economically corresponds to the configured mint or uses USD as its quote currency.

A curator can therefore accidentally pair a mint with a valid but incorrect feed. Supplying the intended price account would brick pricing with `InvalidPriceFeedMint`, while supplying an update matching the incorrect feed would misprice AUM and user settlements.

— RECOMMENDATION

Clearly document this trusted setup assumption. Also consider validating mint/feed pairs during both initialization and vault update.

— RESOLUTION

Status: Acknowledged.

<http://github.com/jup-ag/yield-store/pull/72>

INFORMATIONAL **I-05**

ACKNOWLEDGED

Uninitialized token mint enables share inflation

CATEGORY	Validation
LOCATION	read_mint()
REVIEW	Main Review

— DESCRIPTION

This happens only if a vault is created with an asset mint account that exists but has not been initialized yet

The vault is supposed to list real token mints like USDC, At vault creation, it reads each mint's decimals and stores that forever in AssetConfig.decimals

The issue is that read_mint only checks that the account is owned by SPL Token / Token-2022 and is large enough, It does not check that the mint is initialized

So a blank token-owned mint account with zeroed data is accepted as if it were a real mint, and the vault stores decimals = 0

SPL mint initialization does not require the mint account itself to sign

So after the vault lists that blank mint, an attacker can call Token InitializeMint2 on it, choose themselves as mint authority, choose live decimals, and mint unlimited fake tokens

The code later uses the live mint decimals only for TransferChecked, but values deposits using the old cached decimals in AssetConfig

The concrete flow is that the curator or vault setup accidentally lists an uninitialized token mint

initialize_vault accepts it and stores decimals = 0, the attacker initializes that same mint later and makes themselves the mint authority

The attacker mints fake tokens, In subscribe_direct, the transfer uses the live mint decimals, so the fake token transfer can pass

But valuation uses the cached decimals, so the vault may value tiny or fake raw units as real dollars

I-05 Uninitialized token mint enables share inflation

CONTINUED

The attacker receives real vault shares, then can redeem those shares for genuine vault assets if liquidity and redemption settings allow

This can create shares backed by counterfeit tokens, that can dilute honest users or drain real custody assets through instant redeem or queued redeem fulfillment

— RECOMMENDATION

During vault initialization, reject every asset mint where `is_initialized = false`, and replace unchecked base-layout reads during onboarding with a full extension aware mint unpack

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/73>

INFORMATIONAL I-06

ACKNOWLEDGED

Rounded remint clears unpaid redeem value

CATEGORY Logical Error
LOCATION `cancel_redeem(0`
REVIEW Main Review

DESCRIPTION

This vault has two ways to represent a user's money during a queued redeem

Before fulfillment, the user's shares are burned and the vault records that those burned shares are still owed

After a partial fulfillment, the vault converts those shares into a fixed dollar-like claim called `pending_redeem_value`

The bug happens when only a tiny partial payment is made, then the user cancels

The user's shares are burned immediately when redeem is called, that part is normal

In `fulfill_redeem`, the `FULFILL` delegate can pay whatever idle balance exists in the chosen asset

The only minimum is `pay > 0`, so one tiny raw token unit is enough to create a partial fill

Then `cancel_redeem` converts the unpaid dollar value back into shares using `shares_for_deposit`, which rounds down

If the unpaid value is just below the value of one raw share, this returns 0

The critical mistake is that `cancel_redeem` subtracts the full unpaid liability from `pending_redeem_value` even if it minted 0 shares, then closes the escrow

So the user's remaining claim is erased

Rounding down is acceptable for a new depositor because the vault should not over-mint shares

But this is not a new deposit

This is restoring value that already belonged to the redeemer

The vault already burned their shares and recorded a debt to them

I-06 Rounded remint clears unpaid redeem value

CONTINUED

If it clears that debt, it must give them equivalent shares or keep the leftover debt recorded

The code comments say a partial cancellation should make the user whole, but the implementation can do the opposite

A malicious or compromised FULFILL delegate can choose a listed payout token with only dust available, partially fill with that dust, and leave the user stuck

After the timelock, the user has two bad choices

They can leave the remaining claim stuck because later fills are tied to the first chosen asset

Or they can cancel and lose up to almost one raw share worth of NAV

In the provided example, the user is owed about \$100, receives only \$0.000001, then cancellation mints 0 shares and deletes the remaining \$99.999999 liability

That value becomes extra backing for the remaining shareholders

The timelock, fresh AUM check, PDA checks, and pending_redeem_value reserve do not fix the rounding loss then clear behavior

— RECOMMENDATION

Partial cancellation must use rounding up because it is restoring an existing liability, alternatively, keep any value that cannot be represented by the minted shares recorded as a remaining redeem liability

The escrow must not be closed and the full pending_redeem_value must not be removed unless the user receives equivalent value

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/69>

INFORMATIONAL I-07

ACKNOWLEDGED

Pending redeems hidden by supply saturation

CATEGORY	Validation
LOCATION	Vault::eff_supply(), nav::redeem_owed()
REVIEW	Main Review

DESCRIPTION

The vault must count, live shares: actual Token-2022 share tokens still in circulation, and pending redeem shares: burned shares from users who are waiting to be paid

The correct total is live shares + pending redeem shares

When someone redeems through the queued path, the code burns their share tokens immediately, but the vault still owes them money later

The code knows this and adds pending redeem shares back for fair pricing

The bug is that it adds the two numbers using

```
share_supply.saturating_add(self.pending_redeem_shares.get())
```

saturating_add means that if the real answer is bigger than `u64::MAX`, the code pretends it is exactly `u64::MAX` instead of failing

That is wrong for accounting, If the vault really has more claims than `u64::MAX`, pretending the number is smaller makes every share look more valuable than it really is

The dangerous sequence is that victims queue redemptions, their shares are burned, so the live Token-2022 supply goes down

But the vault still owes them money, tracked as `pending_redeem_shares`, a new depositor mints shares into the live supply space freed by those burns

Token-2022 only checks live supply, so the mint can succeed

Now the real economic supply is live shares + pending redeem shares, and that can exceed `u64::MAX`

`redeem_direct` prices the attacker's shares using the capped supply, so the attacker gets too much money

Because the victims have not been priced yet, `pending_redeem_value` is still zero, so `redeem_direct` reserves nothing for them

I-07 Pending redeems hidden by supply saturation

CONTINUED

The vault can have 100 dollars and 100 claim tickets

50 people hand in tickets and wait for payment, so their tickets are burned but they are still owed money

The vault then sells 50 new tickets

There are now 150 real claims, but the accounting still pretends there are only 100

A new ticket holder can withdraw as if they own 1/100 instead of 1/150, stealing value from the waiting redeemers

An attacker can deposit, receive newly minted shares, immediately redeem_direct, and withdraw more than their fair share

The extra money comes from queued redeemers whose shares were burned but who are not yet protected by pending_redeem_value

It can also make cancel_redeem fail later because the live mint capacity needed to remint victims' burned shares may already be consumed

Deposit caps are optional, instant redeem can be disabled by configuration, and Token-2022 only protects live supply, none of those controls enforce the missing supply invariant

— RECOMMENDATION

Do not use saturation for economic supply accounting, the program must enforce this invariant before every share mint

live share supply + pending_redeem_shares + new shares <= u64::MAX, Any operation that would violate the invariant must revert

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/74>

INFORMATIONAL I-08

ACKNOWLEDGED

Queued deposits settle against stale AUM

CATEGORY	Validation
LOCATION	fulfill()
REVIEW	Main Review

DESCRIPTION

A user puts money into a waiting-room escrow when they use queued Subscribe

Later, a FULFILL delegate decides when that money gets converted into vault shares

The problem is that the user did not approve the final price, minimum shares, deadline, or freshness of the vault's position values

The vault can say its AUM is fresh now even though part of that AUM came from an older cached position value

ValidateAum accepts required position marks if they are still inside their allowed age, then writes

`vault.last_priced_at = now`

After that, Fulfill only checks the fresh vault-level timestamp

It does not check whether every underlying position was freshly revalued

The user thinks they are depositing into the current vault value

But if a vault position already lost value and the cached mark still says it is worth a lot, the user gets too few shares

Their deposit is then added to vault custody, and old shareholders can redeem against the inflated recorded AUM

Subscribe only stores amount, owner, vault, request type, fulfilled flag, and nonce

It stores no `min_shares` and no `deadline`

ValidateAum accepts cached required position marks if they are not stale under the configured age, then stamps the whole vault as priced now

Fulfill requires only the FULFILL permission, checks `vault.require_priced_now(now)`, moves escrowed funds into vault custody, mints shares, then marks the escrow fulfilled

I-08 Queued deposits settle against stale AUM

CONTINUED

RedeemDirect also only checks the aggregate fresh timestamp and can immediately pay from idle liquidity. Suppose the vault claims it has \$100m, but really it only has \$10m because a marked external position crashed.

A victim has \$10m waiting in queued Subscribe.

The FULFILL delegate settles them using the fake \$100m value, so they receive about 10m shares instead of about 100m fair shares.

Their money enters the vault, and old shareholders can redeem using their old shares to withdraw most of the newly added cash.

The main conditions are that a required position mark becomes economically wrong before its freshness window expires, a FULFILL delegate chooses the bad settlement time, the delegate owns shares or colludes with a shareholder, RedeemDirect is enabled, and idle liquidity is available.

Position staleness checks, drift guards, and Pyth freshness help other cases.

But they do not force every required position to be freshly revalued at fulfillment time.

The bug is that queued subscribers accept an unknown future NAV controlled by the fulfiller.

— RECOMMENDATION

A queued subscription must not be an unconditional future market order controlled by the keeper; the request should include user-approved settlement limits such as minimum shares, maximum NAV, a deadline, and acceptable position-mark freshness.

and fulfillment should revert when those limits are not satisfied.

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/54>

INFORMATIONAL **I-09**

ACKNOWLEDGED

Borrowed args create unchecked Rust strings

CATEGORY	Validation
LOCATION	read()
REVIEW	Main Review

— DESCRIPTION

This issue lets attacker controlled instruction bytes reach Rust undefined behavior inside the on-chain program

It's a memory safety issue, this Solana program receives instruction data as raw bytes

In `program/src/lib.rs`, it does `VaultInstruction::deserialize(data)`, then it dispatches to handlers like `InitializeVault` and `UpdateShareMeta`

The risky decode happens in `interface/src/instructions.rs`

```
args: <&InitializeVaultArgs as SchemaRead>::get(reader)?
```

```
args: <&UpdateShareMetaArgs as SchemaRead>::get(reader)?
```

That `&Args` is important, It means borrow the bytes directly as this struct instead of parse and validate every field

Think of the instruction bytes as a form submitted by a user, the safe way is to read each field, check that it is valid, then use it

This code instead takes the whole byte blob, pretends it is already a valid Rust struct, and uses it

That is only safe when every possible byte pattern is valid, but here the struct contains strings

Strings are not any bytes, Rust str must be valid UTF-8, a byte like `0xff` is not valid text

`UpdateShareMetaArgs` contains optional metadata strings

```
PodOption<PodString<25>>
```

```
PodOption<PodString<5>>
```

```
PodOption<PodString<200>>
```

`InitializeVaultArgs` also contains metadata strings

I-09 Borrowed args create unchecked Rust strings

CONTINUED

PodString has a validator that should reject invalid UTF-8, but the borrowed zero-copy decode skips that validator

In `update_share_meta`, the program does

```
if let Some(name) = arguments.metadata_name.get() {  
    value: &name,  
}
```

`value` expects `&str`, so Rust converts `PodString` into `str`

But `PodString` does that using `from_utf8_unchecked`, meaning trust me, these bytes are valid UTF-8, If the attacker sent `0xff`, that trust is false

At that moment, the program has created an invalid Rust `&str`, that is undefined behavior

The same issue exists during vault creation in `initialize_vault`

Normal SDK users probably cannot trigger it because the Rust SDK builds `PodString` from real `&str` and checks length / UTF-8 first

But the SDK is not a security boundary, anyone can submit raw Solana instruction bytes

The reachable path is raw instruction bytes, `VaultInstruction::deserialize`, borrowed zero-copy args, no `PodString` validation

handler converts `PodString` to `&str`, that's undefined behavior

`UpdateShareMeta` requires the vault curator, so it is curator-gated for an existing vault

But `InitializeVault` can be triggered by someone creating their own vault, so external triggerability exists

The impact is that attacker-controlled bytes can cause undefined behavior inside the on-chain program

That is serious because Rust and LLVM are allowed to assume `&str` is always valid UTF-8

Once an invalid `&str` exists, normal safety guarantees no longer apply

In practice, the compiled Solana binary might simply forward bad bytes to `Token-2022` and the CPI may fail

But that is not a real mitigation because the invalid `&str` was already created before the CPI, so it's valid as a memory-safety bug

I-09 Borrowed args create unchecked Rust strings

CONTINUED

— RECOMMENDATION

we need to not deserialize semantic argument structures through wincode's borrowed &T path, make the affected variants own validated arguments and decode them through field-aware SchemaRead implementations

Alternatively, explicitly call ZcValidate before handlers receive them

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/76/>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Duplicate account roles violate borrow safety

CATEGORY	Validation
LOCATION	from_account_unchecked()
REVIEW	Main Review

DESCRIPTION

This issue is about giving the program the same Solana account twice, but under two different names

The strongest real trigger is `delegate = vault_state` in `Fulfill` and `FulfillRedeem`

A Solana instruction passes accounts like this

```
account[1] = delegate
account[2] = vault_state
account[5] = escrow
```

The program assumes those are different accounts, but Solana allows the same account to appear more than once

So an attacker can pass

`delegate = real Vault account V`

`vault_state = same real Vault account V`

Pinocchio represents those as two `AccountViews` pointing to the same account bytes

Rust has a strict rule: if you have `&mut` data, nobody else may read or write that same data through another reference

But `from_account_unchecked` uses `borrow_unchecked()`, which skips runtime borrow tracking

In `fulfill`, the code first creates a mutable `Vault` reference

```
let vault = Vault::read_mut(unsafe { vault_state.borrow_unchecked_mut() }?);
```

Then it reads the `delegate`

```
let del = unsafe { Delegate::from_account_unchecked(delegate) }?;
```

I-10 Duplicate account roles violate borrow safety

CONTINUED

If `delegate` and `vault_state` are the same account, this means the program has `&mut Vault` to `V`

The program then creates shared read access to `V` as `Delegate`, that breaks Rust's safety rules

The `Delegate` check usually fails because a `Vault` account has discriminator 1, while `Delegate` expects discriminator 2

But the failure happens too late, the unsafe shared borrow is created before the discriminator check returns `InvalidAccountData`

So even if the instruction later errors, the program already entered undefined behavior territory

The normal observed result may just be `InvalidAccountData`

The problem is that Rust undefined behavior means the compiler is allowed to assume this aliasing never happens

In optimized SBF, that can theoretically cause stale reads, skipped assumptions, bad state writes, or corrupted accounting

Here that matters because `Fulfill` and `FulfillRedeem` are fund-moving paths

They update AUM, escrow status, redeem liabilities, and perform token mint / transfer CPIs

`delegate = vault_state` in `Fulfill` / `FulfillRedeem` is enough to make the core issue valid

— RECOMMENDATION

Reject duplicate role addresses before unsafe reads, especially `delegate = vault_state`, and replace unchecked borrows with checked `try_borrow` / `try_borrow_mut` guards where practical

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/81>

INFORMATIONAL **I-11**

ACKNOWLEDGED

Bad feed exponent can stall redemptions

CATEGORY	Validation
LOCATION	oracle_value_in_usd()
REVIEW	Main Review

DESCRIPTION

The vault can contain several listed assets

To calculate total vault value, `validate_aum` loops over every asset marked `PRICE_IN_AUM`

It converts each token balance into USD style units using the token decimals, the oracle price, the oracle exponent

The code limits the token decimals, but it does not limit the oracle exponent

The converter first calculates a scaling factor similar to

```
10^abs(exponent + 6 - decimals)
```

If the oracle exponent is too large, this power of ten calculation overflows before the final value can be calculated

This can happen even when the asset balance is zero

The code still constructs the large scaling factor before returning the asset value

An empty asset row should contribute zero to AUM

Instead, it can make the entire vault valuation return `AumOverflow`

Example: the asset uses 18 decimals and the oracle exponent is -27

The resulting scale is $-27 + 6 - 18 = -39$

The converter then attempts to calculate 10^{39}

That value does not fit inside the selected integer type

So the conversion returns `AumOverflow` instead of returning zero for the empty balance

The first deposit follows a special bootstrap path

`subscribe_direct` and `fulfill` can allow the vault's first funding without requiring a complete fresh AUM validation

I-11 Bad feed exponent can stall redemptions

CONTINUED

The vault can therefore become funded while another listed asset row is already poisoned by an unsupported oracle exponent

After bootstrap, normal operation requires `validate_aum` to refresh the vault value

`validate_aum` processes every priced asset row, including the empty poisoned row

Once that conversion fails, the vault cannot obtain a fresh AUM value. Direct deposits stop, direct redemptions stop, queued fulfillment paths stop

A queued redemption can still be requested, but its settlement cannot complete

Users can become unable to exit even when the vault's other custody assets are valid and available

One misconfigured or incompatible listed asset can freeze normal vault operation

The practical recovery requires curator replacing the oracle feed through `update_vault`

But this issue is conditional, It only triggers when a listed asset uses an oracle exponent outside the converter supported range

If every listed asset always remains inside that range, the failure does not occur

— RECOMMENDATION

Validate the complete oracle scaling range when an asset configuration is created or updated

Reject any combination of token decimals and oracle exponent that can make the power of ten calculation overflow

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/79>

INFORMATIONAL **I-12**

ACKNOWLEDGED

Share reentry wrongly charges performance fees

CATEGORY	Logical Error
LOCATION	<code>accrue_fees()</code>
REVIEW	Main Review

— DESCRIPTION

The vault can reach a state where no shares exist but assets still remain accounted inside the vault

share supply = 0, pending redeem shares = 0, pending redeem value = P, AUM = P + G

P is already owed to redeemers

G is free AUM, such as a direct donation later recorded through permissionless `ValidateAum`

There are no active shareholders, but the vault is not treated as empty because `pending_redeem_value` is still greater than zero

If a user then deposits or cancels part of a pending redeem, new shares are created without resetting the performance-fee high-water mark

The code then treats the holderless AUM increase as profit earned by the newly created shares

But that value already existed before the user received those shares

Example: all shares are queued for redemption and partially first-filled

The live share supply becomes zero, the attacker donates G to the vault and calls `ValidateAum`

A victim then deposits D or cancels a remaining redeem claim R

`accrue_fees` sees the old zero share supply

It therefore does not update `hwm_nav`

The next fee calculation treats the pre-existing donation as performance earned by the newly created shares

The charged fee is approximately

I-12 Share reentry wrongly charges performance fees

CONTINUED

$$\text{fee} \approx 10\% \times X \times G / (G + \text{VIRTUAL_ASSETS})$$

X is the new deposit D or the cancelled claim R

With virtual_assets = 1,000, a \$1 donation can create around a \$99.90 performance fee against a \$1,000 deposit

A \$0.01 donation can create around a \$90.91 fee against an approximately \$1,000 cancelled claim

The fee liability reduces the value of the user's newly created shares

Those fees can later be crystallized into shares for the protocol treasury

The user is therefore charged a performance fee on their own principal or on value that existed before they became a shareholder

The issue can affect SubscribeDirect, It can also affect queued Fulfill and partial CancelRedeem

— RECOMMENDATION

Whenever effective share supply changes from zero to a positive amount, set hwm_nav to at least the exact post transition NAV, and do not charge a performance fee for the transition that creates the first new shares

Apply this protection to SubscribeDirect, queued Fulfill, and partial CancelRedeem

— RESOLUTION

Status: Acknowledged.

<https://github.com/jup-ag/yield-store/pull/77>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.